Java Full Stack Development

3. DTOs, Validation and Global Error Handling

Module Topics

- DTOs and the Service Layer
- Bean Validation
- Global Exception Handling
- ControllerAdvice
- Validation Groups

Domain and DTO

- The domain layer
  - JPA entity classes
  - Java objects that are directly mapped to database tables
  - The @Entity annotation tells Hibernate to manage these objects and synchronise them with the database
- Represents the persistent data model
  - It is the database's view of your data

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    private String passwordHash;      // sensitive — must never reach the API
    private String internalStatusCode; // internal — must never reach the API

    @ManyToOne(fetch = FetchType.LAZY)
    private Department department;
}
```

© 2026 by Innovation In Software Corporation

Domain and DTO

- The DTO layer
  - Plain data carrier objects
  - Java records (preferred) or classes
  - Define exactly what the API receives from clients and returns to clients
- Represents the API's view of the data
  - Only the fields that clients are permitted to see or send

```java
public record EmployeeDto(
    Long id,
    String firstName,
    String lastName,
    String departmentName
    // no passwordHash — never exposed
    // no internalStatusCode — never exposed
) {}
```

© 2026 by Innovation In Software Corporation

Domain and DTO

- Returning a JPA entity directly from a REST endpoint instead of a DTO causes four problems
- Uncontrolled serialisation and N+1 queries
  - Jackson (the JSON serialiser) will traverse and serialize the entire object graph of an entity
  - If the Employee has a lazy-loaded Department, and Department has a lazy-loaded list of Employees, Jackson will trigger Hibernate to load everything
  - This happens outside any transaction boundary
  - Problematic if the transaction has already closed
  - The result is either a LazyInitializationException crash or a cascade of unintended database queries

© 2026 by Innovation In Software Corporation

Domain and DTO

- Database schema changes break your API
  - If your entity field is called empSurname in the database and is exposed the entity directly
    - Then the API field is empSurname
  - If the column name changes, then the public API contract also changes
    - Every client breaks
  - With a DTO, the API field name is defined in the DTO
    - Allows the database to change without affecting clients

© 2026 by Innovation In Software Corporation

Domain and DTO

- Sensitive fields are inadvertently exposed
  - The entity exists to represent a database row
  - Database rows contain things clients should never see
    - Password hashes, internal status codes, audit timestamps, soft-delete flags
  - If the entity is returned, Jackson serializes all of unless you remember to annotate every sensitive field with @JsonIgnore
  - A common error is to forget the annotation as the entity evolves

© 2026 by Innovation In Software Corporation

Domain and DTO

- Unexpected database writes
  - Hibernate tracks changes to managed entity instances, called dirty checking
  - If a client sends a JSON request body and it is deserialized directly into a managed @Entity object
  - Then Hibernate may detect the "changes" and write them back to the database at the end of the transaction
  - Even if save() is never called
  - Creates a subtle, dangerous bug that is difficult to reproduce

© 2026 by Innovation In Software Corporation

Service Layer and Translation

- The service layer is where entities and DTOs are translated into each other
- This translation is a deliberate, explicit decision
  - This is not boilerplate to be removed or automated
- Inbound
  - Client sends data → service persists it

```java
// Client sends a CreateEmployeeRequest DTO
// Service maps it to an Employee entity before persisting

@Transactional
public EmployeeDto createEmployee(CreateEmployeeRequest request) {

    // Explicit field-by-field mapping — deliberate decision on every field
    Employee employee = new Employee();
    employee.setFirstName(request.firstName());
    employee.setLastName(request.lastName());
    employee.setDepartmentId(request.departmentId());
    // passwordHash is set here from a separate hashing step — not from the request
    // internalStatusCode is set by business logic — not from the request

    Employee saved = employeeRepository.save(employee);
    return toDto(saved);
}
```

© 2026 by Innovation In Software Corporation

Service Layer and Translation

- Every field that crosses the API boundary is an explicit decision visible in this code
  - If a developer adds a sensitive field to the entity, it does not automatically appear in the API
  - It only appears when a developer makes the conscious choice to add it to the DTO and to the mapping method
- Outbound
  - service reads entity → client receives DTO

```java
// Private mapping method — owned by the service
private EmployeeDto toDto(Employee employee) {
    return new EmployeeDto(
        employee.getId(),
        employee.getFirstName(),
        employee.getLastName(),
        employee.getDepartment().getName()
        // passwordHash deliberately omitted
        // internalStatusCode deliberately omitted
    );
}
```

© 2026 by Innovation In Software Corporation

The Dependency Rule

```
┌──────────────────────────────────────────────────────┐
│  CLIENT (browser, mobile app, another service)       │
└──────────────────────────────────────────────────────┘
                          │
                          │  HTTP  (DTO in, DTO out)
                          ▼
┌──────────────────────────────────────────────────────┐
│  .controller    Knows about: service, dto            │
│                 Knows nothing about: repository, domain │
└──────────────────────────────────────────────────────┘
                          │
                          │  DTO in → service call → DTO out
                          ▼
┌──────────────────────────────────────────────────────┐
│  .service       Knows about: repository, domain, dto │
│                 Knows nothing about: HTTP, controller │
│                 TRANSLATION POINT: entity ↔ DTO      │
└──────────────────────────────────────────────────────┘
                          │
                          │  entity operations
                          ▼
┌──────────────────────────────────────────────────────┐
│  .repository    Knows about: domain                  │
│                 Knows nothing about: HTTP, service logic│
└──────────────────────────────────────────────────────┘
                          │
                          │  SQL / JPA
                          ▼
┌──────────────────────────────────────────────────────┐
│  DATABASE                                            │
└──────────────────────────────────────────────────────┘
```

© 2026 by Innovation In Software Corporation

DTO as Boundary

- The service layer is the mandatory translation point in both directions
  - The domain model never crosses the API surface
  - Each layer has a stable contract that can evolve independently

```
HTTP Request
    |
    ▼
Controller ——— CreateEmployeeRequest (DTO) ———▶ Service
                                                  |
                                          maps to Entity
                                                  |
                                             Repository
                                                  |
                                          maps to Response DTO
                                                  |
                                                  ▼
                                          EmployeeResponse (DTO)
                                                  |
HTTP Response ◀———————————————————————————————————┘
```

© 2026 by Innovation In Software Corporation

DTO Responsibility

- Historical problems with applications that accessed relational databases
  - Tight coupling between the database schema and class structure in the code
    - Using libraries like JDBC
    - SQL becomes part of the application code
  - Created a brittle architecture
    - If the database schema were to change, then code breaks everywhere
  - Code is not written to the logical object model but to the implementation relational model
    - Also locks data storage into a relational model
    - Migrating to another model, like NOSQL breaks client code

Responsibility	Why It Matters
Decouple API schema from domain model	Domain refactors don't break API consumers
Control serialisation explicitly	Every response field is a conscious decision — no accidental exposure
Carry validation constraints	Fail-fast at the boundary before business logic runs
Separate read and write shapes	`CreateRequest` and `Response` evolve independently
Enable API versioning	`v1` and `v2` DTOs coexist without touching the domain

© 2026 by Innovation In Software Corporation

Java Records

- Why records are ideal for DTOs
  - Immutable by default
  - Represents a snapshot, not a live mutable object
  - Compiler generates equals(), hashCode(), toString(), and accessors

```java
// Request DTO — incoming payload, carries validation constraints
public record CreateEmployeeRequest(
    @NotBlank(message = "Name is required")
    @Size(max = 100) String name,

    @NotBlank @Email String email,

    @NotNull
    @DecimalMin(value = "0.01", message = "Salary must be positive")
    BigDecimal salary,

    @NotBlank String departmentCode
) {}

// Response DTO — outbound shape, no validation needed
public record EmployeeResponse(
    Long id,           // server-generated — not in the request
    String name,
    String email,
    BigDecimal salary,
    String departmentCode,
    Instant createdAt  // server-set — not in the request
) {}
```

© 2026 by Innovation In Software Corporation

Bean Validation

- A Java Bean is simply a plain Java object that the framework manages
  - "Validation" means checking that the data it holds meets defined rules
- Bean validation
  - A standard mechanism for attaching and enforcing rules directly on Java objects
  - Has three distinct layers shown on the right
    - Dev writes annotations
    - The spec defines their meaning
    - Validator executes them
    - Spring MVC triggers the execution
    - You never call the validator yourself

Layer	What It Is	Example
Specification	A standard (JSR-380 / Jakarta Validation) that defines the annotation language and the validator contract	Defines what `@NotBlank` means and what a `ConstraintValidator` must implement
Implementation	A library that implements the spec	Hibernate Validator — the reference implementation included in every Spring Boot project
Integration	The framework layer that wires it into the request lifecycle	Spring MVC — calls the validator automatically when `@Valid` is present

© 2026 by Innovation In Software Corporation

Bean Validation

- JSR-380 standardizes
  - The built-in constraint annotations
    - @NotNull, @Size, @Email, etc.
  - @Constraint meta-annotation for defining custom constraints
  - ConstraintValidator<A, T> interface
  - Validator API
    - The programmatic entry point for validation
  - Constraint groups
  - Constraint violation model
  - The cascaded validation model

Layer	What It Is	Example
Specification	A standard (JSR-380 / Jakarta Validation) that defines the annotation language and the validator contract	Defines what `@NotBlank` means and what a `ConstraintValidator` must implement
Implementation	A library that implements the spec	Hibernate Validator — the reference implementation included in every Spring Boot project
Integration	The framework layer that wires it into the request lifecycle	Spring MVC — calls the validator automatically when `@Valid` is present

© 2026 by Innovation In Software Corporation

Bean Validation

- JSR-380 does not standardize
  - How or when validation is triggered
    - That is the framework's responsibility
  - Where constraints should be declared
  - The framework recommends DTOs

Layer	What It Is	Example
Specification	A standard (JSR-380 / Jakarta Validation) that defines the annotation language and the validator contract	Defines what `@NotBlank` means and what a `ConstraintValidator` must implement
Implementation	A library that implements the spec	Hibernate Validator — the reference implementation included in every Spring Boot project
Integration	The framework layer that wires it into the request lifecycle	Spring MVC — calls the validator automatically when `@Valid` is present

© 2026 by Innovation In Software Corporation

Hibernate Validator

- Spring Boot includes Hibernate validator which adds
  - Additional constraints not in the spec:
    - @URL, @CreditCardNumber, @ISBN, @Length, @Range, @UniqueElements
  - Constraint composition
    - Building new constraints from combinations of existing ones
  - Programmatic constraint declaration without annotations
  - Spring integration
    - Recognizing Spring beans as ConstraintValidator implementations
    - Enabling dependency injection into validators

```
spring-boot-starter-validation
    └── hibernate-validator
        └── jakarta.validation-api  (the JSR-380 spec)
```

© 2026 by Innovation In Software Corporation

Bean Validation

- The problem with manual validation
  - Validation logic coupled to the method and is not reusable
  - Constraints are invisible unless you read the entire method body
  - Inconsistencies creep in across different methods
  - Runs inside business logic
    - Validation should never reach this point

```java
// Before Bean Validation — scattered, inconsistent, untestable
public EmployeeResponse create(CreateEmployeeRequest request) {
    if (request.name() == null || request.name().isBlank()) {
        throw new ValidationException("Name is required");
    }
    if (request.email() == null || !request.email().matches(EMAIL_REGEX)) {
        throw new ValidationException("Email is invalid");
    }
    if (request.salary() == null || request.salary().compareTo(BigDecimal.ZERO)
        throw new ValidationException("Salary must be positive");
    }
    // ... actual business logic buried under validation noise
}
```

© 2026 by Innovation In Software Corporation

Bean Validation

- The framework executes validation before the method body runs
  - @Valid on the controller parameter is all that is needed to activate this
  - The service layer receives only valid data
  - Zero validation code in the controller method body

```
HTTP POST /employees
     |
     ▼
Spring MVC deserialises request body → CreateEmployeeRequest
     |
     ▼
@Valid triggers Bean Validation on the DTO
     |
     ├──[constraints pass]──► controller method executes
     |
     └──[constraint fails]──► MethodArgumentNotValidException thrown
                                     |
                                     ▼
                              @ControllerAdvice handles it
                              Returns 400 + structured error payload
```

© 2026 by Innovation In Software Corporation

Bean Validation

- Replacing manual validation with automatic validation

```java
// Constraint declared on the DTO — co-located with the data definition
public record CreateEmployeeRequest(
    @NotBlank(message = "Name is required")
    @Size(max = 100) String name,

    @NotBlank @Email String email,

    @NotNull
    @DecimalMin(value = "0.01", message = "Salary must be positive")
    BigDecimal salary
) {}

// @Valid activates the framework — zero validation logic in the controller
@PostMapping
public ResponseEntity<EmployeeResponse> create(
        @Valid @RequestBody CreateEmployeeRequest request) {

    // Reaching this line means ALL constraints on `request` have passed
    return ResponseEntity.status(201).body(service.create(request));
}
```

© 2026 by Innovation In Software Corporation

Built-in Constraint Annotations

Annotation	Validates	Example
`@NotNull`	Field is not null (any type)	`@NotNull Long id`
`@NotBlank`	String is not null AND not whitespace	`@NotBlank String name`
`@Size(min, max)`	String or Collection length	`@Size(min=2, max=100)`
`@Email`	Syntactically valid email	`@Email String email`
`@Min` / `@Max`	Integer/long numeric range	`@Min(0) int quantity`
`@DecimalMin` / `@DecimalMax`	BigDecimal range	`@DecimalMin("0.01")`
`@Pattern(regexp)`	Regex match	`@Pattern(regexp="[A-Z]{3}")`
`@Future` / `@Past`	Date relative to now	`@Future LocalDate startDate`
`@Valid`	Cascade into a nested object	`@Valid AddressRequest address`

© 2026 by Innovation In Software Corporation

@NotNull vs @NotBlank

- A common mistake
  - For user-provided text fields
    - Always use @NotBlank
  - For non-String types
    - Long, Integer, BigDecimal
    - Use @NotNull
    - Blank is not a concept
  - Use @NotNull on a String
    - Will silently accept " " as valid input
    - A common source of data quality bugs

	`@NotNull`	`@NotBlank`
`null`	✗ FAILS	✗ FAILS
`""` (empty string)	✓ PASSES	✗ FAILS
`"  "` (whitespace only)	✓ PASSES	✗ FAILS
`"Alice"`	✓ PASSES	✓ PASSES

© 2026 by Innovation In Software Corporation

Custom Validators

- Built-in constraints handle syntax
  - Business rules represent semantic understanding of the business domain
- Examples of semantic constraints
  - Does this department code exist in the database?
  - Is this username already taken?
  - Is this product SKU valid for the selected region?
- Custom validators implement these checks in the Bean Validation framework:
  - Declared on the DTO alongside built-in constraints
  - Execute automatically as part of @Valid processing
  - Can be Spring beans with injected dependencies

```java
// Step 1: Define the annotation
@Target({ ElementType.FIELD })
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = DepartmentCodeValidator.class)
public @interface ValidDepartmentCode {
    String message() default "Unknown department code";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

// Step 2: Implement the validator — it's a Spring bean
@Component
public class DepartmentCodeValidator
        implements ConstraintValidator<ValidDepartmentCode, String> {

    private final DepartmentRepository departmentRepository;

    public DepartmentCodeValidator(DepartmentRepository repo) {
        this.departmentRepository = repo;
    }

    @Override
    public boolean isValid(String code, ConstraintValidatorContext ctx) {
        if (code == null) return true; // let @NotNull handle null separately
        return departmentRepository.existsByCode(code);
    }
}
```

© 2026 by Innovation In Software Corporation

Custom Validators

- Validators run during the validation phase
  - Before the service transaction starts
- Race condition risk
  - A department code might exist when the validator checks, then be deleted before the save completes
  - Never rely on a custom validator as a substitute for a database constraint (FOREIGN KEY, UNIQUE)
  - Use both: the validator for fast, user-friendly feedback; the DB constraint as the authoritative guard

```
// Step 3: Use it exactly like any built-in constraint
public record CreateEmployeeRequest(
    @NotBlank String name,
    @ValidDepartmentCode String departmentCode  // semantic, database-backed ch
) {}
```

© 2026 by Innovation In Software Corporation

Custom Validators

- Performance impact
  - Validators run on every request
  - A validator making an expensive query adds latency to every inbound call
  - Keep validator queries simple: existence checks only
  - Move complex cross-entity validation into the service layer, inside the transaction

```
// Step 3: Use it exactly like any built-in constraint
public record CreateEmployeeRequest(
    @NotBlank String name,
    @ValidDepartmentCode String departmentCode  // semantic, database-backed ch
) {}
```

© 2026 by Innovation In Software Corporation

Global Exception Handling

- In any layered application
  - Exceptions travel up through the call stack
- Without a strategy, each layer makes its own decision
  - The repository logs and re-throws
  - The service catches, wraps, and re-throws
  - The controller catches and formats differently from every other controller
  - The client receives something different from every endpoint
- The real question is not how to handle exceptions but where to handle it

```
Repository throws DataAccessException
        |
        ▼
Service receives it — does it belong here?
        |
        ▼
Controller receives it — does it belong here?
        |
        ▼
Spring MVC — what does the client receive?
```

© 2026 by Innovation In Software Corporation

Global Exception Handling

- Two distinct responsibilities
  - 1: Detection
    - Something went wrong
    - An exception was thrown somewhere in the call stack
    - The system needs to know about it
  - 2: Translation
    - The exception must become an HTTP response
    - The response must have the right status code
    - The response body must follow the API error contract
    - No internal detail should leak to the client
  - Detection happens where the failure occurs
    - Translation happens in one centralized location.

```
Repository throws DataAccessException
        |
        ▼
Service receives it — does it belong here?
        |
        ▼
Controller receives it — does it belong here?
        |
        ▼
Spring MVC — what does the client receive?
```

© 2026 by Innovation In Software Corporation

Global Exception Handling

- "Global" means
  - One place translates exceptions into HTTP responses
    - Regardless of which controller threw them
  - One place enforces the error response contract
    - Every error has the same shape
  - One place decides what information the client receives and what stays internal
  - Every controller is free from error translation logic entirely

```
Repository throws DataAccessException
        |
        ▼
Service receives it — does it belong here?
        |
        ▼
Controller receives it — does it belong here?
        |
        ▼
Spring MVC — what does the client receive?
```

© 2026 by Innovation In Software Corporation

Exception Handling Anti-Pattern

- Without a centralized strategy
  - Distributed translation across end points
- Multiply this across 20 endpoints
  - 20 different error shapes
  - Stack traces leaking in some, suppressed in others
  - Status codes inconsistent
    - One endpoint returns 409, another returns 422 for the same condition
  - No way to audit error behaviour across the API surface
  - Adding a new exception type requires touching every controller

```java
@PostMapping
public ResponseEntity<?> createEmployee(@RequestBody EmployeeRequest req) {
    try {
        return ResponseEntity.ok(service.create(req));
    } catch (DepartmentNotFoundException e) {
        return ResponseEntity.status(404)
            .body(Map.of("error", e.getMessage()));     // shape 1
    } catch (DuplicateEmailException e) {
        return ResponseEntity.status(409)
            .body("Duplicate: " + e.getMessage());      // shape 2 — plain string
    } catch (Exception e) {
        return ResponseEntity.status(500)
            .body(Map.of("message", "Something went wrong",
                         "trace", e.getStackTrace()));  // shape 3 — leaks stack tr
    }
}
```

© 2026 by Innovation In Software Corporation

Exception Handling Anti-Pattern

- Client view
  - When error responses are assembled per-controller
  - The client cannot write reliable error handling
  - An inconsistent error contract is a broken contract
  - The client cannot fulfill its side of the API agreement if the server cannot hold its side

```
// Client code — forced to handle every possible shape
async function createEmployee(data) {
    try {
        const response = await api.post('/employees', data);
        return response.data;
    } catch (error) {
        // Which shape did we get this time?
        if (typeof error.response.data === 'string') {
            // came from endpoint A
        } else if (error.response.data.error) {
            // came from endpoint B
        } else if (error.response.data.message) {
            // came from endpoint C — but wait, is .trace also present?
        }
        // This code grows with every new endpoint and every new exception type
    }
}
```

© 2026 by Innovation In Software Corporation

Mental Model

- Errors are a first-class feature
  - A common misconception: errors are edge cases
- The production reality
  - Roughly 20–30% of HTTP responses in a well-used API are errors
  - Every client that calls your API must handle every error type you produce
  - Your error contract is as important as your success contract
  - Clients write code against your errors just as they write code against your responses

Old thinking	Correct thinking
Errors are exceptional — handle them where they occur	Errors are a first-class part of the API contract
Each controller knows best how to report its own errors	One mechanism owns the translation, consistently
The client will figure out the error shape	The error shape is specified and versioned like any other response
Stack traces are useful to return — they help debugging	Stack traces are internal diagnostic information — never for clients

© 2026 by Innovation In Software Corporation

Error Contract

- A structured error response
  - A promise the server makes to every client
  - The client can rely on
    - Status always matches the HTTP status code
    - Code is a stable, machine-readable string and safe to switch on in client code
    - Message is a human-readable description suitable for logging
    - Path identifies which endpoint produced the error
    - Timestamp enables correlation with server-side logs
    - Errors is always present: empty array for non-validation errors, populated for 400s

```
{
  "status": 422,
  "code": "BUSINESS_RULE_VIOLATION",
  "message": "An employee with this email already exists",
  "path": "/api/v1/employees",
  "timestamp": "2025-03-22T10:15:00Z",
  "errors": []
}
```

© 2026 by Innovation In Software Corporation

Exception Types

- The exception type
  - Carries the semantic meaning of the failure
  - The handler translates that semantic meaning into HTTP terms
  - The controller and service never need to know what HTTP status code their exceptions produce
    - They throw the right semantic exception
    - The handler decides the HTTP mapping
    - These are separate concerns.

```
// Each exception type is a named category of failure
class EntityNotFoundException extends RuntimeException { }
class BusinessRuleException extends RuntimeException { }
class InsufficientScopeException extends RuntimeException { }
class TokenExpiredException extends RuntimeException { }
class ConcurrentModificationException extends RuntimeException { }
```

```
EntityNotFoundException     → 404  NOT_FOUND
BusinessRuleException       → 422  BUSINESS_RULE_VIOLATION
InsufficientScopeException  → 403  INSUFFICIENT_SCOPE
TokenExpiredException       → 401  TOKEN_EXPIRED
ConcurrentModificationException → 409  CONFLICT
```

© 2026 by Innovation In Software Corporation

Exception Hierarchy

- Well-designed exception hierarchy
  - Makes the handler predictable and maintainable
  - The handler can map at the category level (NotFoundException = 404) without listing every subtype
  - New specific exceptions inherit the right HTTP mapping automatically
  - The hierarchy documents the failure taxonomy of the application

```
RuntimeException
├── ApplicationException          ← base for all application exceptions
│   ├── NotFoundException         → 404
│   │   ├── EntityNotFoundException
│   │   └── ResourceNotFoundException
│   ├── BusinessRuleException     → 422
│   │   ├── DuplicateEmailException
│   │   └── InsufficientBalanceException
│   ├── ConflictException         → 409
│   └── AuthorisationException    → 403
│       ├── InsufficientScopeException
│       └── TokenExpiredException
└── Exception (catch-all)         → 500
```

```java
// Handler maps the category — all subtypes are covered
@ExceptionHandler(NotFoundException.class)
ResponseEntity<ErrorResponse> handleNotFound(NotFoundException ex, WebRequest req) {
    return ResponseEntity.status(404)
        .body(ErrorResponse.of(404, "NOT_FOUND", ex.getMessage(), req, List.of()));
}
// EntityNotFoundException and ResourceNotFoundException both match — no extra handl
```

© 2026 by Innovation In Software Corporation

@ExceptionHandler

- Has a specific and important purpose
  - Handles anything not caught by a more specific handler
  - Guarantees the client always receives a structured response and never a raw Spring error page
  - Guarantees stack traces never reach the client

```
@ExceptionHandler(Exception.class)
ResponseEntity<ErrorResponse> handleUnexpected(Exception ex, WebRequest req) {
    log.error("Unhandled exception on {} {}", req.getDescription(false), ex.getClass().getName(), ex);
    return ResponseEntity.internalServerError()
        .body(ErrorResponse.of(500, "INTERNAL_ERROR", "An unexpected error occurred", req, List.of()));
}
```

© 2026 by Innovation In Software Corporation

@ExceptionHandler

- Has a specific and important purpose
  - What it is not
    - A substitute for specific handlers
    - A place to put business logic
    - An indication that something is working correctly
- If the catch-all fires frequently in production then
  - Either a new exception type needs its own handler
  - Or something unexpected is failing regularly and needs investigatio

```
@ExceptionHandler(Exception.class)
ResponseEntity<ErrorResponse> handleUnexpected(Exception ex, WebRequest req) {
    log.error("Unhandled exception on {} {}", req.getDescription(false), ex.getClass().getName(), ex);
    return ResponseEntity.internalServerError()
        .body(ErrorResponse.of(500, "INTERNAL_ERROR", "An unexpected error occurred", req, List.of()));
}
```

© 2026 by Innovation In Software Corporation

Separation of Layers

- Global exception handling
  - Enforces the layered architecture
  - A controller method that only expresses the happy path is correctly designed
  - The controller has
    - No try-catch
    - No ResponseEntity construction for error cases
    - No status code decisions
    - No error message formatting

Layer	Its job	What it does with exceptions
Repository	Data access	Throws `DataAccessException` subtypes for infrastructure failures
Service	Business logic	Throws semantic exceptions (`EntityNotFoundException`, `BusinessRuleException`) for business failures
Controller	HTTP binding	Throws nothing — delegates to service, lets exceptions propagate
`@ControllerAdvice`	Error translation	Catches everything, translates to HTTP, enforces the error contract

```java
@PostMapping
public ResponseEntity<EmployeeResponse> create(
        @Valid @RequestBody CreateEmployeeRequest request) {
    // If service throws, @ControllerAdvice handles it
    // This method only expresses the happy path
    return ResponseEntity.status(201).body(service.create(request));
}
```

© 2026 by Innovation In Software Corporation

@ControllerAdvice

- One @ControllerAdvice class
  - Handles exceptions from all controllers in the application
  - Registered at startup
  - Zero per-controller configuration required
  - Extending ResponseEntityExceptionHandler
  - Intercepts Spring MVC's own exceptions
    - Including MethodArgumentNotValidException

```
Exception thrown anywhere in any Controller
      |
      ▼
Spring MVC scans registered @ControllerAdvice classes
      |
      ▼
Finds the @ExceptionHandler method with the most specific matching type
      |
      ▼
Invokes the handler method
      |
      ▼
Handler returns a ResponseEntity → that becomes the HTTP response
      |
No exception ever reaches the client directly
```

© 2026 by Innovation In Software Corporation

@ControllerAdvice

- Controller
  - It applies to the controller layer specifically
  - It intercepts what happens around @Controller and @RestController classes
- Advice
  - Aspect-Oriented Programming (AOP) term
  - It means "code that runs around other code without the other code knowing about it"

```
Exception thrown anywhere in any Controller
      |
      ▼
Spring MVC scans registered @ControllerAdvice classes
      |
      ▼
Finds the @ExceptionHandler method with the most specific matching type
      |
      ▼
Invokes the handler method
      |
      ▼
Handler returns a ResponseEntity → that becomes the HTTP response
      |
No exception ever reaches the client directly
```

© 2026 by Innovation In Software Corporation

@ControllerAdvice

- @ControllerAdvice
  - AOP advice that applies globally across all controllers
  - Without any controller needing to reference it, import it, or know it exists
  - The controller does not call the advic
  - Spring MVC calls it automatically when an exception escapes a controller method

```
Without AOP thinking:
____________________

ControllerA
  try { ... }
  catch { format error }

ControllerB
  try { ... }
  catch { format error }

ControllerC
  try { ... }
  catch { format error }
```

```
With @ControllerAdvice:
_______________________

ControllerA
  happy path only

ControllerB
  happy path only

GlobalExceptionHandler
  all error formatting
  in one place
```

© 2026 by Innovation In Software Corporation

@ControllerAdvice

- AOP
  - *"Some concerns like logging, security, transaction management, error handling, need to apply consistently across many classes, but should not be written into those classes directly"*
  - Writing cross-cutting concerns into every class
    - Creates repetition
    - Couples unrelated responsibilities
    - Makes the concern inconsistent as the codebase grows
  - AOP externalises the concern into a separate Aspect and weaves it in automatically

```
Without AOP thinking:
____________________

ControllerA
  try { ... }
  catch { format error }

ControllerB
  try { ... }
  catch { format error }

ControllerC
  try { ... }
  catch { format error }
```

```
With @ControllerAdvice:
_______________________

ControllerA
  happy path only

ControllerB
  happy path only

GlobalExceptionHandler
  all error formatting
  in one place
```

© 2026 by Innovation In Software Corporation

@ControllerAdvice

- Startup sequence
  - There is no wiring to do
  - No registration call
  - No reference in any controller
  - The scan finds it; the framework uses it.
  - Startup shown on the right

```java
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component          // ← makes it a Spring bean, discovered by component scan
public @interface ControllerAdvice {
    // scope-limiting attributes — discussed shortly
}
```

```
Application starts
      |
      ▼
Component scan finds @ControllerAdvice class
      |
      ▼
Spring instantiates it as a singleton bean
      |
      ▼
Spring MVC's ExceptionHandlerExceptionResolver reads it
      |
      ▼
Registers all @ExceptionHandler methods in an internal lookup table
      |
      ▼
When any controller throws — the lookup table is consulted
```

© 2026 by Innovation In Software Corporation

@ControllerAdvice

- It can do three things
- Exception handling: @ExceptionHandler
  - Intercepts exceptions thrown from controller methods and converts them to responses
- Model attrb binding: @ModelAttribute
  - Adds attributes to every model before any controller method runs
  - Used in MVC non-REST applications.

```
@ExceptionHandler(EntityNotFoundException.class)
ResponseEntity<ErrorResponse> handleNotFound(EntityNotFoundException ex) { ... }
```

```
@ModelAttribute("currentUser")
UserContext currentUser(Authentication auth) {
    return userContextService.from(auth);
}
```

© 2026 by Innovation In Software Corporation

@ControllerAdvice

- Data binding init: @InitBinder
  - Configures data binding for every controller
  - Can block specific fields from being bound from requests.
- In REST APIs, @ExceptionHandler is the dominant use
  - The others apply in traditional MVC applications with rendered views.

```
@ModelAttribute("currentUser")
UserContext currentUser(Authentication auth) {
    return userContextService.from(auth);
}
```

© 2026 by Innovation In Software Corporation

@ControllerAdvice

- @ControllerAdvice classes often extend
  - ResponseEntityExceptionHandler
  - Spring MVC throws its own internal exceptions for standard failure conditions
  - Without ResponseEntityExceptionHandler, these produce Spring's default error responses
  - Allows intercepting these by overriding specific methods
  - Then the defined error contract applies to every failure, including framework-level ones

Spring MVC Internal Exception	Condition	Default behaviour without extending
`MethodArgumentNotValidException`	`@Valid` fails	Spring's own error format
`HttpMessageNotReadableException`	JSON cannot be parsed	Spring's own error format
`HttpRequestMethodNotSupportedException`	Wrong HTTP method	Spring's own error format
`MissingServletRequestParameterException`	Required `@RequestParam` absent	Spring's own error format
`NoHandlerFoundException`	No matching endpoint	Spring's own error format

© 2026 by Innovation In Software Corporation

Override Example

```java
@ControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    // Spring MVC calls this when @Valid fails
    // Without overriding: Spring returns its own format
    // With overriding: you return your error contract format
    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex,
            HttpHeaders headers,
            HttpStatusCode status,
            WebRequest request) {

        var errors = ex.getBindingResult().getFieldErrors().stream()
            .map(fe -> new FieldError(fe.getField(), fe.getRejectedValue(), fe.getDefaultMessage()))
            .toList();

        return ResponseEntity.badRequest()
            .body(ErrorResponse.of(400, "VALIDATION_FAILED",
                "Request validation failed", request, errors));
    }

    // Spring MVC calls this when JSON body cannot be parsed
    @Override
    protected ResponseEntity<Object> handleHttpMessageNotReadable(
            HttpMessageNotReadableException ex,
            HttpHeaders headers,
            HttpStatusCode status,
            WebRequest request) {

        return ResponseEntity.badRequest()
            .body(ErrorResponse.of(400, "MALFORMED_REQUEST",
                "Request body could not be parsed", request, List.of()));
    }
}
```

© 2026 by Innovation In Software Corporation

Handler Lookup

- When an exception escapes a controller
  - Spring MVC runs a resolution algorithm
- The resolution always selects the most specific match in the hierarchy
  - A handler for every specific exception subtype
  - Map at the category level and new subtypes are handled automatically

```
Exception thrown: DepartmentNotFoundException
(which extends EntityNotFoundException, which extends RuntimeException)

Step 1: Search @ExceptionHandler methods in the @ControllerAdvice class
        for DepartmentNotFoundException
        → Not found

Step 2: Search for the immediate supertype: EntityNotFoundException
        → Found: handleNotFound(EntityNotFoundException ex)
        → Use this handler

Step 3: (would continue up the hierarchy if step 2 also failed)
        EntityNotFoundException → RuntimeException → Exception
```

```
// If both of these exist, DepartmentNotFoundException uses the first one
@ExceptionHandler(DepartmentNotFoundException.class)
ResponseEntity<ErrorResponse> handleDeptNotFound(...) { ... }  // ← selected

@ExceptionHandler(EntityNotFoundException.class)
ResponseEntity<ErrorResponse> handleNotFound(...) { ... }  // ← not selected
```

© 2026 by Innovation In Software Corporation

Scoping

- By default
  - @ControllerAdvice applies to all controllers
  - It can be scoped
- When scoping is useful
  - An application serves both a REST API and a rendered web UI
    - Errors should be formatted differently for each
  - API v1 and v2 need different error contracts during a migration period
  - An internal admin interface needs different error handling from the public API
- In a pure REST API with a single error contract, global scope (no scoping) is correct

```java
// Apply only to controllers in a specific package
@ControllerAdvice("com.example.api.v1")

// Apply only to controllers in listed packages
@ControllerAdvice(basePackages = {"com.example.api.v1", "com.example.api.v2"})

// Apply only to controllers in packages containing these classes
@ControllerAdvice(basePackageClasses = {EmployeeController.class, DepartmentController.class})

// Apply only to controllers with a specific annotation
@ControllerAdvice(annotations = RestController.class)

// Apply only to specific controller types
@ControllerAdvice(assignableTypes = {EmployeeController.class})
```

© 2026 by Innovation In Software Corporation

Ordering

- An application can have more than one @ControllerAdvice class
  - Spring searches all @ControllerAdvice classes in @Order sequence
  - Within each class, most specific exception type wins
  - First match across all classes is used and subsequent classes are not consulted

```java
@ControllerAdvice
@Order(1)                    // higher priority
public class SecurityExceptionHandler {
    @ExceptionHandler(AuthorisationException.class)
    ResponseEntity<ErrorResponse> handleAuth(...) { ... }
}

@ControllerAdvice
@Order(2)                    // lower priority
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {
    @ExceptionHandler(EntityNotFoundException.class)
    ResponseEntity<ErrorResponse> handleNotFound(...) { ... }

    @ExceptionHandler(Exception.class)
    ResponseEntity<ErrorResponse> handleUnexpected(...) { ... }
}
```

© 2026 by Innovation In Software Corporation

@ControllerAdvice

- What it does NOT do
  - It does not catch exceptions from @Service or @Repository directly
    - It only intercepts exceptions that have propagated up through a controller method
    - If a service method is called outside a controller context (e.g. from a CommandLineRunner, a Kafka consumer, or a scheduled task), @ControllerAdvice has no effect
  - It does not apply to filter-level exceptions
    - Spring Security filters, OncePerRequestFilter implementations, and other servlet filters run before Spring MVC
    - Exceptions thrown there bypass @ControllerAdvice entirely
    - They need separate handling in the filter itself or via an AuthenticationEntryPoint / AccessDeniedHandler
  - It does not replace a proper exception hierarchy
    - @ControllerAdvice is the translation layer and does not remove the need to throw well-typed exceptions from service code
    - A single catch-all handler without specific handlers produces vague 500 responses where 404s and 422s should appear
  - It does not handle async exceptions automatically
    - Exceptions from @Async methods or reactive pipelines require separate handling strategies

© 2026 by Innovation In Software Corporation

Summary

- @ControllerAdvice
  - Spring-managed singleton
  - Discovered by component scan
  - Registers its @ExceptionHandler methods as global interceptors across all controllers
  - Translating any exception that escapes a controller method into a structured HTTP response, without any controller knowing the handler exists

```
     HTTP Request arrives
             |
┌────────────▼────────────┐
│      Controller         │
│   (happy path only)     │
│  calls service(...)     │
└─────────────────────────┘
             | exception propagates up
┌────────────▼────────────┐
│       Service           │
│   throws semantic       │
│   exception             │
└─────────────────────────┘
             | exception propagates up
┌────────────▼────────────┐
│  Spring MVC intercepts  │
│  the escaping exception│
└─────────────────────────┘
             | consults handler registry
┌────────────▼────────────┐
│   @ControllerAdvice     │
│  GlobalExceptionHandler│
│                         │
│  finds matching         │
│  @ExceptionHandler      │
│  method                 │
│                         │
│  returns                │
│  ResponseEntity         │
└─────────────────────────┘
             |
┌────────────▼────────────┐
│  Structured HTTP        │
│  Error Response         │
│  (consistent contract) │
└─────────────────────────┘
```

© 2026 by Innovation In Software Corporation

Validation Groups

- The problem
  - The same data, but three different contracts:
    - On POST: name, email, salary, departmentCode are all mandatory
    - On PUT: same as POST — full replacement requires all fields
    - On PATCH: any subset of fields — whatever the client wants to change
  - If you apply @NotBlank to every field unconditionally
    - POST works correctly
    - PUT works correctly
    - PATCH is impossible: the client cannot send a partial update without failing validation on the absent fields

```
POST /employees        → Create a new employee      (all fields required)
PUT  /employees/{id}   → Replace an employee        (all fields required)
PATCH /employees/{id}  → Partially update           (only changed fields sent)
```

© 2026 by Innovation In Software Corporation

Validation Groups

- The problem
  - The same data, but three different contracts:
    - On POST: name, email, salary, departmentCode are all mandatory
    - On PUT: same as POST — full replacement requires all fields
    - On PATCH: any subset of fields — whatever the client wants to change
  - If you remove @NotBlank from every field
    - PATCH works
    - POST and PUT accept incomplete records resulting in missing required fields reaching the service layer
  - Neither extreme works
    - Validation Groups are the resolution

```
POST /employees        → Create a new employee      (all fields required)
PUT  /employees/{id}   → Replace an employee        (all fields required)
PATCH /employees/{id}  → Partially update           (only changed fields sent)
```

© 2026 by Innovation In Software Corporation

Validation Groups

- A validation group is a marker interface
  - No methods
  - No fields
  - No logic
  - No annotations
  - Only purpose is to serve as a type token
    - A unique identity in the Java type system that the validation framework can use as a label

```
public interface OnCreate {}
public interface OnUpdate {}
public interface OnPatch  {}
```

© 2026 by Innovation In Software Corporation

Validation Groups

- Using the first shown form on the right
  - "This constraint belongs to the OnCreate group"
- Using the second form on the right
  - "Evaluate only the constraints in the OnCreate group".
- The interface is the label
  - The framework does the rest

```
@NotBlank(groups = OnCreate.class)
String name;
```

```
@NotBlank(groups = OnCreate.class)
String name;
```

© 2026 by Innovation In Software Corporation

The Default Group

- Every constraint that has no groups attribute belongs to a built-in group called Default
- Activating a specific group
  - In the example
    - @Email and @Size are silently skipped
    - This may not be what was intended

```
public record EmployeeRequest(
    @NotBlank(groups = OnCreate.class) String name,   // belongs to OnCreate
    @Email String email,                              // belongs to Default
    @Size(max = 100) String name                      // belongs to Default
) {}
```

```
// This activates OnCreate constraints only
@Validated(OnCreate.class) @RequestBody EmployeeRequest req
```

```
Constraints evaluated:
  @NotBlank(groups = OnCreate.class) on name  ✓  — in OnCreate
  @Email on email                             ×  — in Default, not activated
  @Size(max = 100) on name                    ×  — in Default, not activated
```

© 2026 by Innovation In Software Corporation

The Default Group

- The fix
  - Explicitly include Default

```
@Validated({OnCreate.class, Default.class}) @RequestBody EmployeeRequest req
```

```
Constraints evaluated:
  @NotBlank(groups = OnCreate.class) on name  ✓  – in OnCreate
  @Email on email                             ✓  – in Default, now activated
  @Size(max = 100) on name                    ✓  – in Default, now activated
```

© 2026 by Innovation In Software Corporation

The Default Group

- What the developer expects
  - name required, email validated, salary required and positive
- What actually happens
  - Response: 201 Created
  - Validation passed
  - @Email and @DecimalMin were never evaluated
  - The invalid email and negative salary reached the service layer

```java
public record CreateEmployeeRequest(
    @NotBlank(groups = OnCreate.class)
    String name,

    @Email                          // ← Default group — developer forgot to group
    String email,

    @NotNull(groups = OnCreate.class)
    @DecimalMin(value = "0.01")     // ← Default group — developer forgot to group
    BigDecimal salary
) {}

@PostMapping
public ResponseEntity<EmployeeResponse> create(
        @Validated(OnCreate.class) @RequestBody CreateEmployeeRequest req) {
    return ResponseEntity.status(201).body(service.create(req));
}
```

```
POST /employees
{
  "name": "Alice",
  "email": "not-an-email",
  "salary": -500.00
}
```

© 2026 by Innovation In Software Corporation

Ordering Validation Execution

- Sometimes the order in which groups are evaluated matters
  - Check that departmentCode is not blank (syntactic check)
  - Only if it is not blank, check that it exists in the database (expensive semantic check)
  - Without ordering, both run simultaneously
  - The database query fires even when the field is empty
- @GroupSequence defines an ordered execution

```java
@GroupSequence({SyntaxChecks.class, SemanticChecks.class})
public interface FullValidation {}

public interface SyntaxChecks {}
public interface SemanticChecks {}

public record CreateEmployeeRequest(
    @NotBlank(groups = SyntaxChecks.class)
    @ValidDepartmentCode(groups = SemanticChecks.class)
    String departmentCode
) {}

// Controller activates the sequence
@PostMapping
public ResponseEntity<EmployeeResponse> create(
        @Validated(FullValidation.class) @RequestBody CreateEmployeeRequest req) { .
```

© 2026 by Innovation In Software Corporation

Ordering Validation Execution

- What happens
  - Step 1:
    - Evaluate SyntaxChecks group
    - @NotBlank runs
    - If it fails, stop here
    - No database query fired
  - Step 2:
    - Only if Step 1 passed)
    - Evaluate SemanticChecks group
    - @ValidDepartmentCode runs
    - Database query fires
    - Only reached if the field was not blank

```java
@GroupSequence({SyntaxChecks.class, SemanticChecks.class})
public interface FullValidation {}

public interface SyntaxChecks {}
public interface SemanticChecks {}

public record CreateEmployeeRequest(
    @NotBlank(groups = SyntaxChecks.class)
    @ValidDepartmentCode(groups = SemanticChecks.class)
    String departmentCode
) {}

// Controller activates the sequence
@PostMapping
public ResponseEntity<EmployeeResponse> create(
        @Validated(FullValidation.class) @RequestBody CreateEmployeeRequest req) { .
```

© 2026 by Innovation In Software Corporation

Groups vs Separate DTOs

- Use validation groups when
  - The fields across operations are identical or near-identical
  - Only the activation of constraints differs, not the structure
  - You have two or three groups at most
  - The operations are closely related (create/update/patch on the same resource)

```java
// Too many groups — this DTO is doing too much
public record EmployeeRequest(
    @NotBlank(groups = {OnCreate.class, OnUpdate.class})
    @Null(groups = {OnPatch.class, OnImport.class, OnMigration.class})
    String name,

    @NotNull(groups = OnCreate.class)
    @Null(groups = {OnUpdate.class, OnPatch.class})
    Long originalSystemId,

    @Valid(groups = {OnCreate.class, OnImport.class})
    @Null(groups = {OnUpdate.class, OnPatch.class, OnMigration.class})
    AddressRequest address
) {}
```

© 2026 by Innovation In Software Corporation

Groups vs Separate DTOs

- Use separate DTOs when:
  - Create and update have meaningfully different fields, not just different constraints
  - The operations have different shapes that would require many nullable fields on a shared DTO
  - You have more than three groups — complexity signals the DTO is serving too many contexts
- Readability matters more than deduplication

```java
// Too many groups — this DTO is doing too much
public record EmployeeRequest(
    @NotBlank(groups = {OnCreate.class, OnUpdate.class})
    @Null(groups = {OnPatch.class, OnImport.class, OnMigration.class})
    String name,

    @NotNull(groups = OnCreate.class)
    @Null(groups = {OnUpdate.class, OnPatch.class})
    Long originalSystemId,

    @Valid(groups = {OnCreate.class, OnImport.class})
    @Null(groups = {OnUpdate.class, OnPatch.class, OnMigration.class})
    AddressRequest address
) {}
```

© 2026 by Innovation In Software Corporation

The Complete Picture

- Groups do not replace the other validation mechanisms
  - They compose with them

Concept	What it means
Group = marker interface	No logic — just a type token used as a label
Default group is not automatic	Constraints without `groups` are skipped unless `Default.class` is explicitly activated
Groups compose	Sequences order them; multiple groups activate combined sets; null-pass behaviour determines what runs on optional fields

```java
public record CreateEmployeeRequest(

    // Always validated (Default group), required on create and update
    @NotBlank(groups = {OnCreate.class, OnUpdate.class})
    @Size(max = 100)                                // Default group — always checkec
    String name,

    // Format always validated; presence required on create only
    @Email                                          // Default group
    @NotBlank(groups = OnCreate.class)
    String email,

    // Syntactic check always; semantic check in sequence after syntax
    @NotBlank(groups = {OnCreate.class, SyntaxChecks.class})
    @ValidDepartmentCode(groups = SemanticChecks.class)
    String departmentCode,

    // Required on create; forbidden on update (client cannot change it)
    @NotNull(groups = OnCreate.class)
    @Null(groups = OnUpdate.class)
    @DecimalMin(value = "0.01", groups = OnCreate.class)
    BigDecimal initialSalary

) {}
```

© 2026 by Innovation In Software Corporation

Questions?